Welcome

Digital and Computer Vision

Lesson 3: Session 2

Feature Detection with OpenCV

Summary of the previous session: Image segmentation and object extraction

In the previous session we have covered :

Watershed algorithm for image segmentation.
K-means clustering for color segmentation.
GrabCut for object extraction

Summary of the current session: Feature Detection with OpenCV

In this session we will covered the following:

What are features on images?
Detecting key points using Harris Corner Detection.
Feature matching (ORB, SIFT, and SURF).

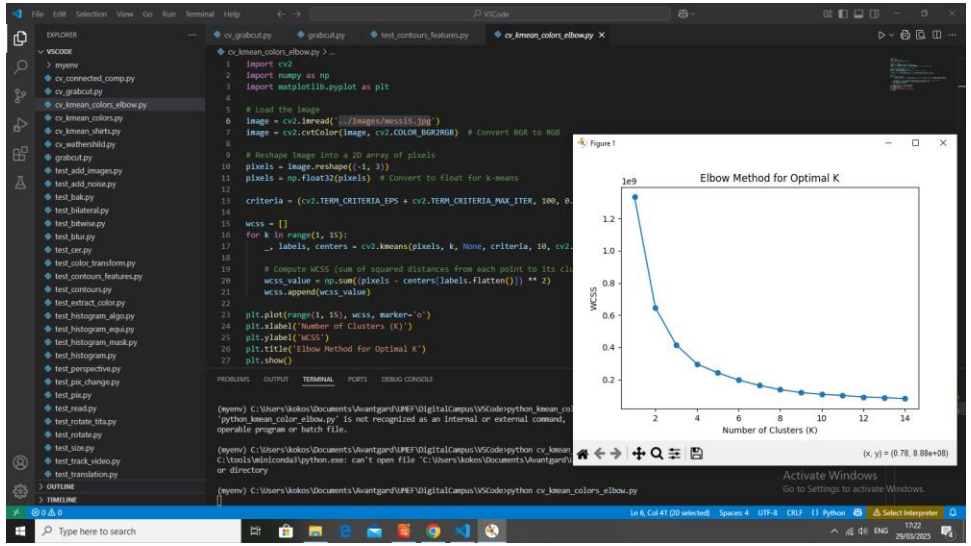
Exercise 1: from previous session

Modify the code of K-Means Clustering for Color Segmentation to compute the within-cluster sum of squares (WCSS) for K 1 to 15 and plot the results in a matplotlib.pyplot plot.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
image = cv2.imread('../Images/messi5.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  # Convert BGR to RGB

# Reshape image into a 2D array of pixels
pixels = image.reshape((-1, 3))
pixels = np.float32(pixels)  # Convert to float for k-means
```

Exercise 1: from previous session

```
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)

wcss = []
for k in range(1, 15):
    _, labels, centers = cv2.kmeans(pixels, k, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)
    # Compute WCSS (sum of squared distances from each point to its cluster center)
    wcss_value = np.sum((pixels - centers[labels.flatten()]) ** 2)
    wcss.append(wcss_value)

plt.plot(range(1, 15), wcss, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal K')
plt.show()
```



Exercise 2 : from previous session code example

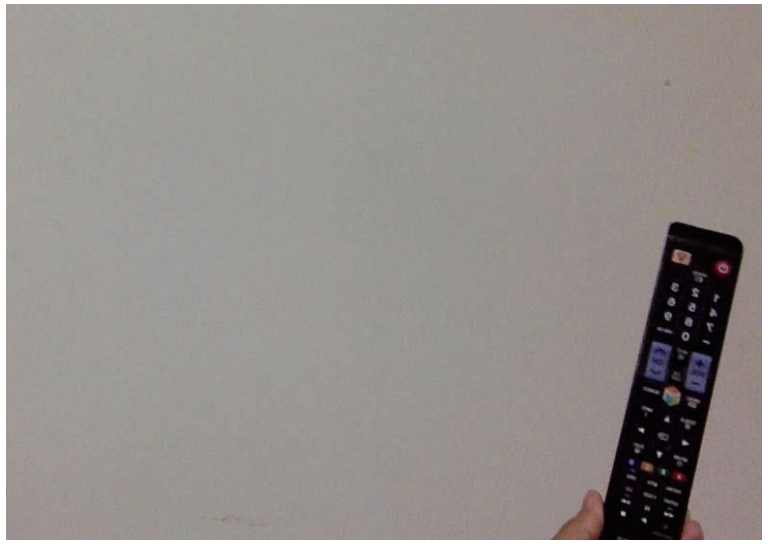
Get the interactive tool using grabcut, grabcut.py, from github. You can refine the grabCut result (step 5 of the algorithm) by use of this interactive tool.

Let’s demonstrate this iteratively in Visual Studio Code

Intuitive approach to features on images

The human eye is more sensitive to high frequency content as compared to low frequency content. Let's look at the image on the right. If you notice, your eye immediately went to the TV remote, even though it's not at the center of the image. We automatically tend to gravitate towards the interesting regions in the image because that is where all the information is. This is what our brain needs to store in order to recollect it later.

When we build object recognition systems, we need to detect these "interesting" regions to create a signature for the image. These interesting regions are characterized by keypoints. This is why keypoint detection is critical in many modern computer vision systems.



Jodseph Howse, Prateek Joshi, Michael Beyeler, Open CV Computer Vision Project with Python, Published by Packt Publishing Ltd. 2016

What are features on images?

In the previos setion we have defined **features** as measurable properties or characteristics of the data used for classification or prediction. Here we will go more deeply in this definition:

Image features are small patches of images or image propertiews that contains characteristic properties of objects in the scene. An image feature is usually composed of a feature keypoint and a feature descriptor.
The **keypoint** usually contains the patch 2D position and other information if available such as scale and orientation.
The **descriptor** contains the vector with mathematical description of the patch and is used to compare the similarity between image features.

Features are the backbone of computer vision. Whether using traditional methods like SIFT, HOG, and SURF or deep learning-based CNN, features help machines "understand" images.

Types of Features in Computer Vision 1/2

1. **Low-Level Features** : Basic image properties like color, texture, and edges (we already know from in image analysis).
Edges – Capturing lines with sudden intensity changes.
Corners – Points with significant intensity variation in multiple directions.
Blobs – Regions of interest detected based on intensity variations.
Texture – Patterns of intensity variation in an image.

2. **Local Features** : Local features describe distinctive regions in an image that are computed at specific keypoints detected using sophisticated algorithms.
SIFT (Scale-Invariant Feature Transform) .
SURF (Speeded-Up Robust Features).
ORB (Oriented FAST and Rotated BRIEF).
FAST (Features from Accelerated Segment Test).
BRIEF (Binary Robust Independent Elementary Features).

https://docs.opencv.org/4.x/df/d54/tutorial_py_features_meaning.html

Types of Features in Computer Vision 2/2

3. **Global Features** : These features describe the entire image rather than specific points.
Histogram-based features – Used for color-based image recognition.
Shape descriptors – Used for object recognition based on shape.
HOG (Histogram of Oriented Gradients) – Used for detecting objects like pedestrians.

4. **Deep Learning-Based Features** : Deep learning extracts high-level features automatically using convolutional neural networks (CNNs).
CNN Feature Maps – Learned feature representations from convolutional layers.
ResNet/VGG Features – Pretrained deep learning models that generate powerful features.
Autoencoders – Used to learn compressed representations of images.

Feature workflow - Detection, Description and Matching

Feature detection, description and matching are fundamental steps in computer vision tasks.

1. Feature detection is the process of identifying features in an image.

2. Once identified we have to assigning a numerical descriptor (a feature vector) to each detected feature. This descriptor encodes important properties so that it can be matched across different images.

3. Finaly, we can compare feature descriptors between two images using feature matching algorithms.

Applications of Features in Computer Vision

1. **Object Recognition** – Identify objects using feature matching (e.g., SIFT, HOG).
2. **Image Stitching** – Match keypoints between images to create panoramas.
3. **Motion Tracking** – Track objects in video (e.g., optical flow).
4. **Facial Recognition** – Detect and recognize faces based on features.
5. **3D Reconstruction** – Extracting 3D information from images.
6. **Augmented Reality (AR)** – Detecting key features in real-world scenes for AR overlays.

Harris Corner Detection

Harris Corner Detection is an algorithm used to identify corner points in an image. A corner is a point where intensity changes in multiple directions, making it a highly distinguishable and stable feature for object detection and tracking.

It was introduced by Chris Harris and Mike Stephens in 1988 and is based on analyzing how pixel intensities change in a small neighborhood.

How Does Harris Corner Detection Work? 1/2

The Harris Corner Detector works by computing the rate of intensity change in different directions at each pixel. The key steps are:

1. Compute Image Gradients. Calculate the gradient (derivatives) of the image in the x and y directions using Sobel filters :

$$I_x = \frac{\partial I}{\partial x}, \quad I_y = \frac{\partial I}{\partial y}$$

2. **Construct the Structure Tensor (Second-Moment Matrix)**. For each pixel, build a matrix based on gradient values:

$$M = \sum_{window} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

How Does Harris Corner Detection Work? 2/2

3. Compute Corner Response Function. The corner strength is determined using the determinant and trace of M :

$$R = \det(M) - k \cdot (\mathrm{trace}(M))^2$$

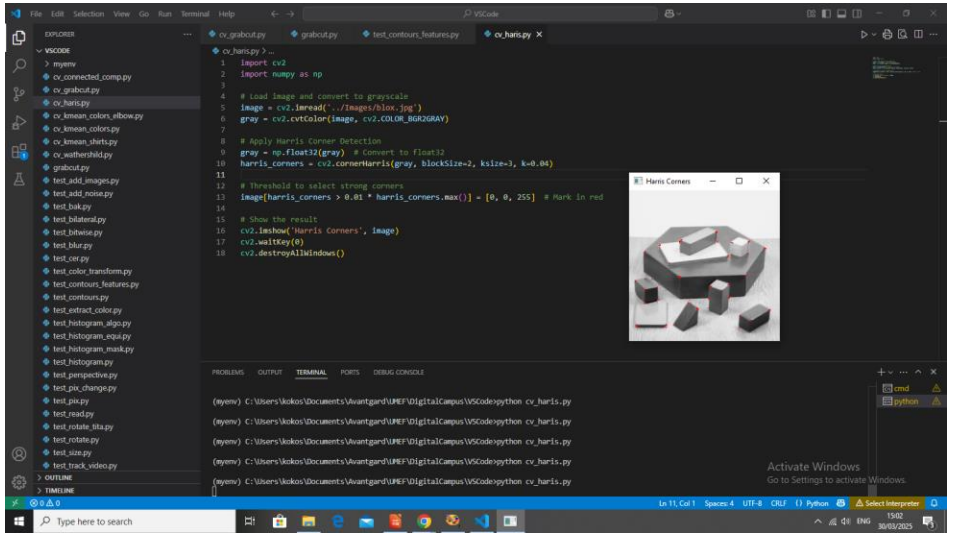
where:

- $\det(M) = I_x^2 I_y^2 - (I_x I_y)^2$
- $\mathrm{trace}(M) = I_x^2 + I_y^2$
- k is an empirical constant (typically **0.04 - 0.06**).

4. **Thresholding and Non-Maximum Suppression.** Identify corners where R exceeds a threshold. In a small local neighborhood (e.g., a 3×3 or 5×5 window), keep only the pixel with the highest R and suppress others.

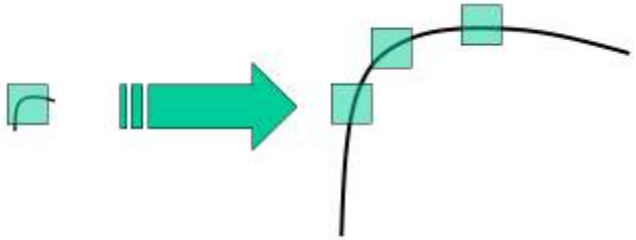
Harris Corner Detection code example

```
import cv2
import numpy as np

# Load image and convert to grayscale
image = cv2.imread('../Images/blox.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Harris Corner Detection
gray = np.float32(gray)  # Convert to float32
harris_corners = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)

# Threshold to select strong corners
image[harris_corners > 0.01 * harris_corners.max()] = [0, 0, 255]

# Show the result
cv2.imshow('Harris Corners', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Scale invariance problem

The Harris corner detector is rotation-invariant, which means, even if the image is rotated, we can find the same corners. However Harris corner is not scale invariant. For example in the image below a corner within a small window is flat when it is zoomed in the same window.

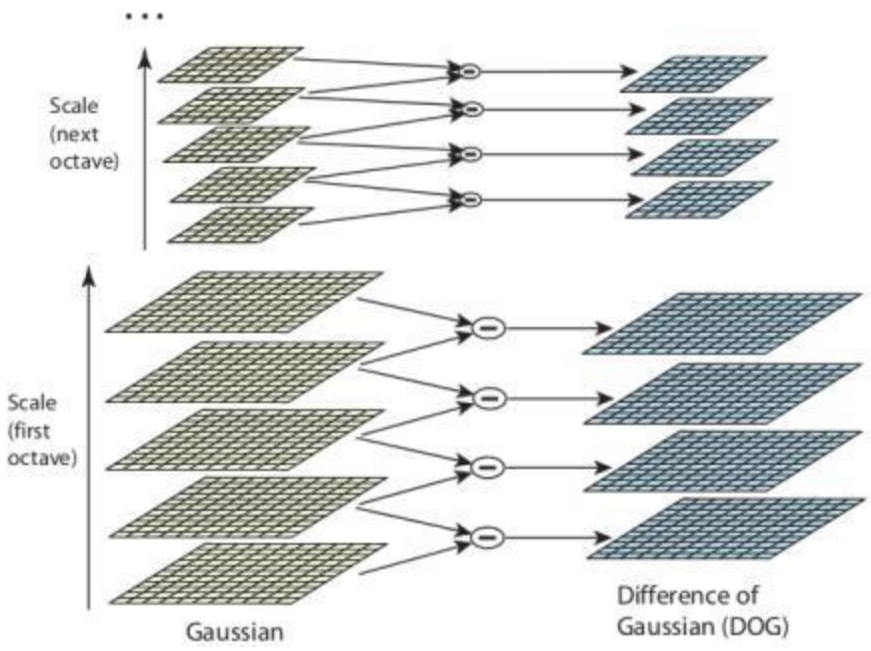


https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html

In the second case, the detector will not pick up this corner. And, since it was picked up in the original image, the second image will not be matched with the first one. It's basically the same image, but the corner features based method will totally miss it.

Scale-Invariant Feature Transform (SIFT)

To make scale invariant transform SIFT algorithm uses Difference of Gaussians, obtained as the difference of Gaussian blurring of an image with two different sigma. This process is done for different octaves of the image in Gaussian Pyramid. It is represented in below image:



https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html

SIFT algorithm

1. **Scale-Space Construction (Finding Keypoints at Different Scales)**. To make SIFT scale-invariant, the image is blurred with Gaussian filters at multiple scales. A pyramid of images is created by applying Gaussian blur and downsampling at different levels. This produces a scale-space representation where key features are identified at multiple scales.

2. **Difference of Gaussian (DoG) for Keypoint Detection**. The Difference of Gaussians (DoG) is calculated by subtracting two blurred images at different scales.

3. **Keypoint Localization (Filtering Out Weak Points)**. Keypoints are refined using a mathematical technique called Taylor series expansion to eliminate unstable points. Keypoints with low contrast or on edges are discarded to keep only strong features.

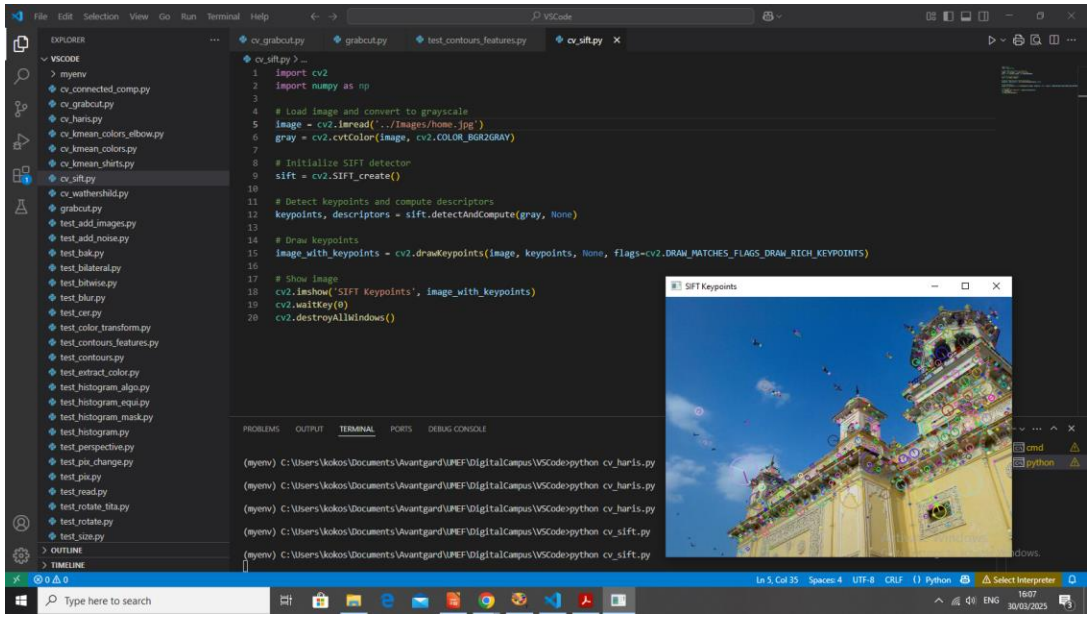
4. **Orientation Assignment (Rotation Invariance).** Each keypoint is assigned an orientation based on the gradient directions in its local region. This allows the descriptor to be rotation-invariant.

5. **Keypoint Descriptor Calculation**. A 128-dimensional descriptor is computed for each keypoint. The descriptor is created using histograms of gradient magnitudes and orientations. This ensures robustness against noise, lighting, and viewpoint changes.

SIFT code example

SIFT was initialy patented, but the patent expired in the year 2020. So now is free of use.

```
import cv2

# Load image and convert to grayscale
image = cv2.imread('../Images/home.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Initialize SIFT detector
sift = cv2.SIFT_create()
# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)
# Draw keypoints
image_with_keypoints = cv2.drawKeypoints(image, keypoints, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
# Show image
cv2.imshow('SIFT Keypoints', image_with_keypoints)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Speeded-Up Robust Features (SURF)

SURF (Speeded-Up Robust Features) is a feature detection and description algorithm designed as a faster alternative to SIFT. It detects keypoints and extracts descriptors for tasks like object recognition, image matching, and tracking.

SURF improves over SIFT by using Hessian-based keypoint detection in the place of Gausian differences and integral images to speed up computation.

Hessian Matrix

The Hessian matrix is a square matrix of second-order partial derivatives of a function. It is used in image processing and feature detection (e.g., SURF) to determine regions of interest like corners and blobs. For a function $f(x,y)$, the Hessian matrix is:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

The determinant of the Hessian matrix helps locate points of interest efficiently and remains invariant to scale changes.

If Det(H) > 0 a key point (blob) is detected
If Det(H) < 0 likly an edge detected
If Det(H) = 0 flat region

SURF uses the Hessian determinant to locate keypoints efficiently. Instead of directly computing second derivatives (which is slow), it uses box filters to approximate them, speeding up processing.

Box Filters

Box filters are integral image-based filters that approximate second-order Gaussian derivatives efficiently. They allow quick computation of feature responses across multiple scales. The filters resemble Haar wavelets and come in different sizes (e.g., 9×9, 15×15, 21×21, etc.).

A box filter for computing the second derivative in the X-direction looks like this:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

The left side (-1 values) detects decreasing gradients.

The right side (+1 values) detects increasing gradients.

The center (0 values) ensures the response is only sensitive to edges.

Integral Images

An Integral Image (also called a Summed-Area Table) is a data structure that allows fast computation of sum, mean, or other features in a rectangular region of an image. The integral image at a pixel (x,y) is defined as the sum of all pixel intensities above and to the left of (x,y):

$$I_{\text{integral}}(x, y) = \sum_{i=0}^{x} \sum_{j=0}^{y} I(i, j)$$

Where:

- $I_{\text{integral}}(x, y)$ is the integral image value at (x, y).
- $I(i, j)$ is the pixel intensity at (i, j).

Computing the sum of pixel values in a region of an image is slow using traditional summation. With integral images, the sum of any rectangular region is computed in constant time (O(1)), regardless of size. For a rectangle defined by four points A, B, C, D, the sum of pixels inside it is computed as:

$$\text{Sum} = I_{\text{integral}}(D) - I_{\text{integral}}(B) - I_{\text{integral}}(C) + I_{\text{integral}}(A)$$

How Does SURF Work?

1. **Keypoint Detection (Using Hessian Matrix)**. Detects keypoints using the Hessian matrix determinant, which identifies strong edges and corners efficiently.Instead of using a Difference of Gaussians (DoG) like SIFT, it applies box filters (approximating second-order Gaussian derivatives) to speed up calculations.

2. **Orientation Assignment**. To ensure rotation invariance, SURF computes the dominant orientation by analyzing the Haar wavelet responses around each keypoint. The highest response direction is assigned as the keypoint's orientation.

3. **Descriptor Calculation**. Instead of SIFT's 128-dimensional descriptor, SURF uses a more compact 64-dimensional descriptor, making it faster. It divides the local region into 4x4 subregions and calculates Haar wavelet responses in x and y directions.

SURF code example

Unfortunatly the SURF patent will expire in 2029 therefore we can not run this code in the free version of OpenCV

```
import cv2

# Load image and convert to grayscale
image = cv2.imread('../Images/home.jpg', cv2.IMREAD_GRAYSCALE)
# Initialize SURF detector (requires OpenCV contrib module)
surf = cv2.xfeatures2d.SURF_create(400)  # Threshold for keypoints
# Detect keypoints and compute descriptors
keypoints, descriptors = surf.detectAndCompute(image, None)
# Draw keypoints
image_with_keypoints = cv2.drawKeypoints(image, keypoints, None, (255, 0, 0), 4)

# Show result
cv2.imshow('SURF Keypoints', image_with_keypoints)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Oriented FAST and Rotated BRIEF (ORB)

ORB is an efficient, patent-free alternative to SIFT and SURF. ORB is widely used in image matching, object recognition, and SLAM (Simultaneous Localization and Mapping). ORB came from "OpenCV Labs". This algorithm was brought up by Ethan Rublee, Vincent Rabaud, Kurt Konolige and Gary R. Bradski in their paper ORB: An efficient alternative to SIFT or SURF in 2011.

ORB Algorithm

ORB is a combination of two algorithms: FAST and BRIEF

1. Feature Detection – FAST (Features from Accelerated Segment Test). Detects keypoints by analyzing pixel intensity changes. ORB improves FAST by adding a multi-scale pyramid and computing the Harris corner measure to filter strong keypoints.

2. Feature Description – BRIEF (Binary Robust Independent Elementary Features)
ORB modifies BRIEF to be rotation-invariant. Uses an orientation-based rotation matrix to align keypoints.

3. Feature Matching – Hamming Distance. ORB uses a binary descriptor, making matching faster using Hamming distance.

How FAST Works ?

1. **Pixel Selection**: FAST works by evaluating a circle of 16 pixels around a candidate pixel in an image.

2. **Segment Test**: For each pixel in the circle, the algorithm checks if the surrounding pixels are significantly brighter or darker than the center pixel. It tests whether there is a continuous segment of pixels in the circle that is either all brighter or all darker than the center pixel.

3. **Thresholding**: If N pixels from the circle are either all brighter or all darker than the center pixel by a threshold T, the center pixel is considered a corner. N can be set as a parameter.

4. **Corner Detection**: Once a corner is found, it is marked as a keypoint.

FAST is very fast because it only needs to check a small number of pixels (16 in the circle) around each candidate, making it highly efficient, particularly for real-time processing.

How BRIEF Works ?

BRIEF is a feature descriptor, it doesn't provide any method to find the features. BRIEF works by analyzing the neighborhood around each keypoint and generating a binary string (bit vector) that encodes whether one pixel in the neighborhood is brighter than another pixel. This is done through a series of comparisons between pixel intensities.

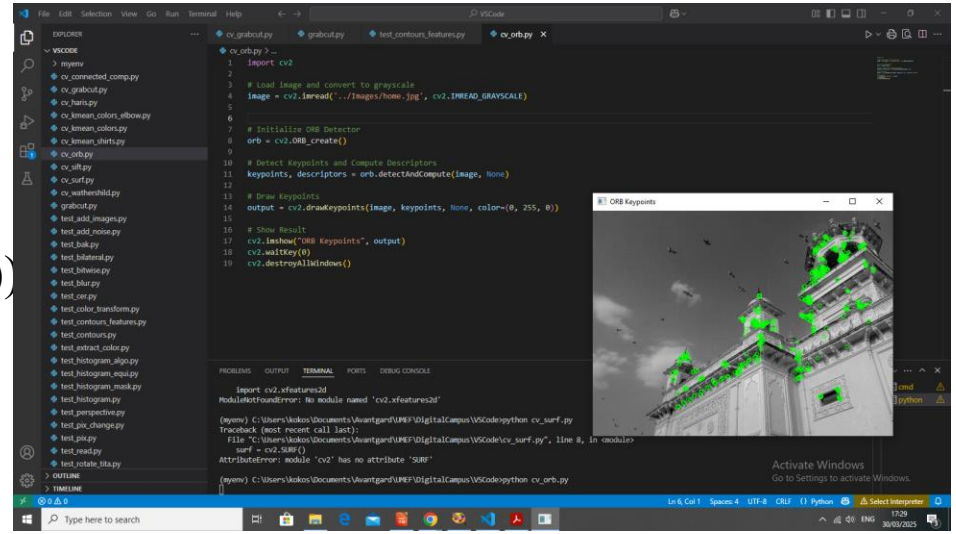
Once descriptors are computed, you can match keypoints between two images by comparing their binary descriptors.

This comparison is very fast since we just need to compare the binary strings bit by bit, typically using Hamming distance.

Hamming distance measures the number of differing bits between two binary strings. The keypoints with the smallest Hamming distance are considered the best matches.

ORB code example

```
import cv2

# Load image and convert to grayscale
image = cv2.imread('../Images/home.jpg', cv2.IMREAD_GRAYSCALE)

# Initialize ORB Detector
orb = cv2.ORB_create()

# Detect Keypoints and Compute Descriptors
keypoints, descriptors = orb.detectAndCompute(image, None)

# Draw Keypoints
output = cv2.drawKeypoints(image, keypoints, None, color=(0, 255, 0))

# Show Result
cv2.imshow("ORB Keypoints", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Comparison between ORB, SIFT and SURF

Feature	ORB	SIFT	SURF
Speed	🚀 Fast	🐢 Slow	🐢 Slow
Rotation Invariance	✅ Yes	✅ Yes	✅ Yes
Scale Invariance	✅ Yes	✅ Yes	✅ Yes
Patent-Free	✅ Yes	❌ No	❌ No
Feature Descriptor	Binary (Fast Matching)	Float (Slow Matching)	Float (Slow Matching)
Best For	**Real-time applications**	High-precision matching	Robust feature matching

Feature Matching

Feature matching identifies corresponding points (features) between two images. These features, detected by methods like ORB and described by descriptors such as BRIEF, help establish correspondences. Once keypoints and descriptors are extracted, the goal is to match them to align, stitch, or compare the images

Brute-Force Matcher

Brute Force Matcher (BFMatcher) is a simple and straightforward method for matching feature descriptors between two images. It works by comparing each descriptor from one image with every descriptor from the second image and finding the best match based on a given distance metric.

How BFMatcher Works ?

1. **Descriptor Comparison**: BFMatcher compares every descriptor in the first set to all descriptors in the second set.

2. **Distance Calculation**: The distance between descriptors is computed using an appropriate distance metric:

For SIFT/SURF (floating-point descriptors): Euclidean distance (cv2.NORM_L2).

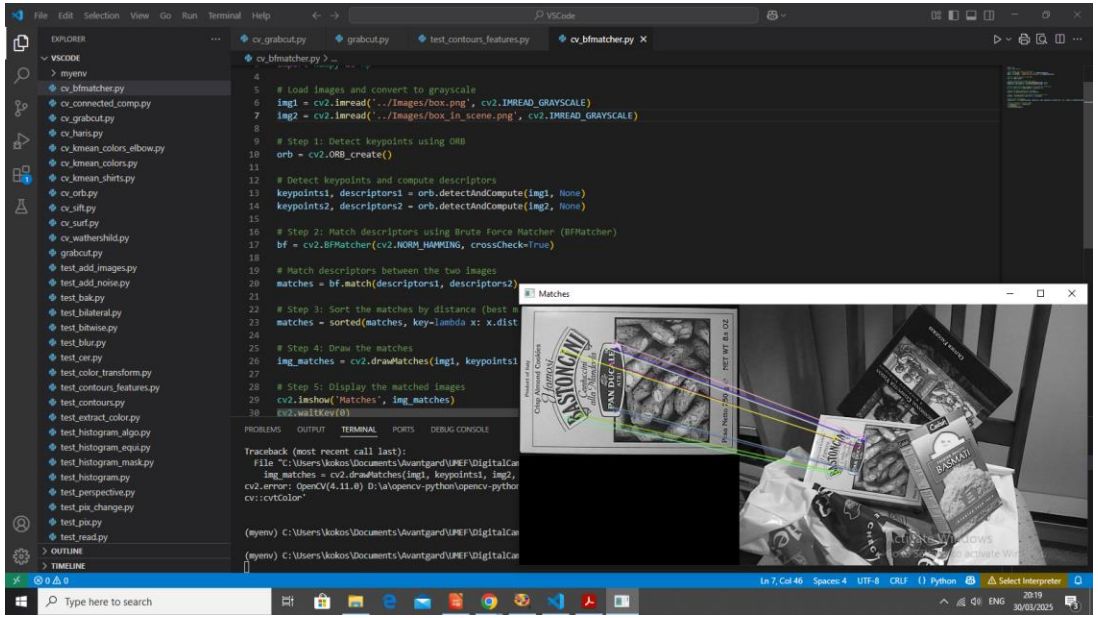
For ORB/BRIEF (binary descriptors): Hamming distance (cv2.NORM_HAMMING).

3. **Best Matches Selection**: Based on the calculated distances, the closest descriptor matches are selected.

Brute-Force Matcher example code

```
import cv2
import numpy as np

# Load images and convert to grayscale
img1 = cv2.imread('../Images/box.png', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('../Images/box_in_scene.png', cv2.IMREAD_GRAYSCALE)

# Step 1: Detect keypoints using ORB
orb = cv2.ORB_create()

# Detect keypoints and compute descriptors
keypoints1, descriptors1 = orb.detectAndCompute(img1, None)
keypoints2, descriptors2 = orb.detectAndCompute(img2, None)

# Step 2: Match descriptors using Brute Force Matcher (BFMatcher)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
```

Brute-Force Matcher example code




```
# Match descriptors between the two images
matches = bf.match(descriptors1, descriptors2)

# Step 3: Sort the matches by distance (best matches first)
matches = sorted(matches, key=lambda x: x.distance)

# Step 4: Draw the matches
img_matches = cv2.drawMatches(img1, keypoints1, img2, keypoints2, matches[:10], None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

# Step 5: Display the matched images
cv2.imshow('Matches', img_matches)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Exercises

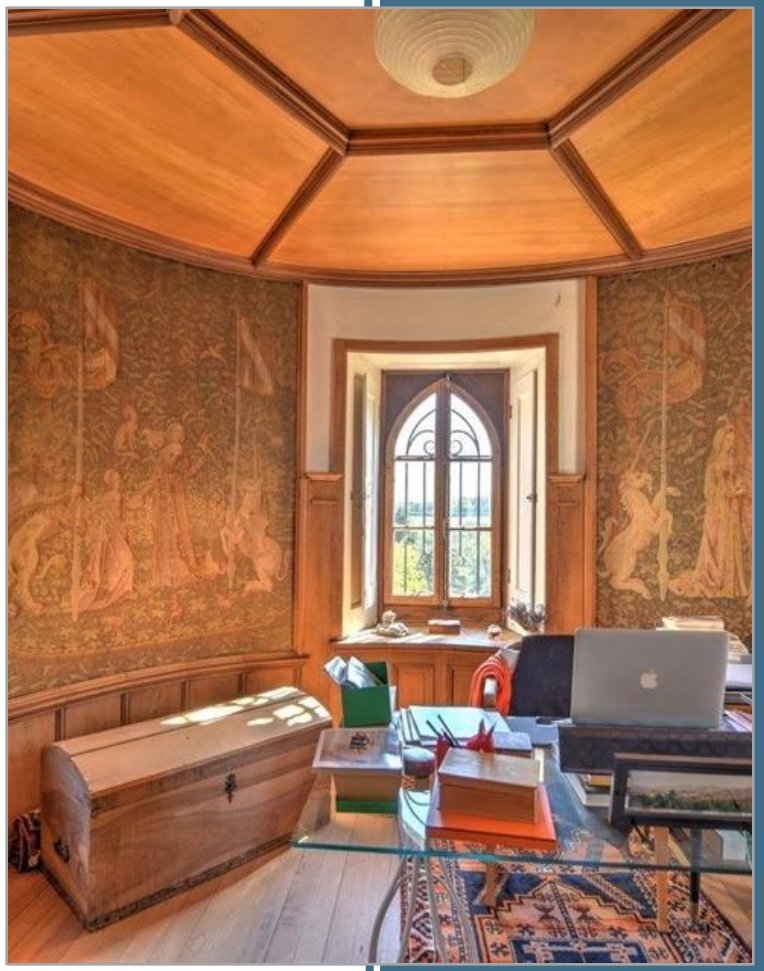
In the book of Jodseph Howse, Prateek Joshi, Michael Beyeler, “Open CV Computer Vision Project with Python”, read the Chapter 5: Creating a Panoramic Image. Explain and run the code for creating panoramic images from this chapter

https://hlevkin.com/hlevkin/45pythonImageProc/Howse-Joshi-Beyeler_opencv_computer_vision_projects_with_python.pdf

In this session we have covered:

Feature Detection with OpenCV
What are features on images?
Detecting key points using Harris Corner Detection.
Feature matching (ORB, SIFT, and SURF).

In the next session we will covered:

Object Detection with Haar Cascades & YOLO
Understanding Haar Cascades for face detection.
Introduction to YOLO and real-time object detection.

THANKS !!!